

WFM × Finance Integration

Implementation Report

FastAPI Backend · Agent-Level Financial Model · v0.1

Field	Value
Source design	WFM_Finance_Integration_Design.pdf v0.1
Stack	FastAPI 0.115 / Python 3.11 / SQLAlchemy 2.0 / Pydantic 2.9
Storage	SQLite (local) - swap to Postgres via DATABASE_URL
Tests	26 / 26 passing
HTTP routes	33 endpoints
Frontend	Java compatible via CORS + OpenAPI client gen
Data source	Agent_Breakdown_140426.csv (streaming ingest)
Report generated	2026-05-22 23:19

Every section of the source design document is mapped to a concrete module, endpoint, and test case in this implementation. The full coverage matrix is on the last page.

1. Executive Summary

A stateless FastAPI service that consumes the WFM hourly breakdown and produces a per-agent financial breakdown of the same period. It owns the formulas; it never owns hours (WFM does) or rates (HR/Contracts does). The Billing Engine is a pure function over fully-hydrated inputs, which is the property that makes it testable, reproducible, and replayable for audit.

Three transport paths cover every realistic ingest mode:

Path	Driven by	Use case
POST /finance/breakdown	WFM cron at end-of-shift	Push model - the canonical section 7.1 contract
POST /ingest/csv/*	Java frontend or scheduled batch	Bulk pull from Agent_Breakdown_140426.csv
GET /ingest/csv/stream	Java frontend EventSource	Live progress feed via Server-Sent Events

Java-frontend compatibility comes from three things: (1) CORS allowlist pre-configured for typical JVM dev ports, (2) snake_case JSON matching the PDF contract verbatim, (3) an auto-generated OpenAPI 3 spec at /openapi.json that the openapi-generator CLI converts to typed Java DTOs.

2. Architecture (PDF Section 3)

Five components and three data stores; one deterministic chain.

```

WFM hourly breakdown --> Leave Ledger lookup --> Rate Card lookup
                    --> Billing Engine --> Per-agent financial record
                                   |
                                   +--> AuditLog (hash + provenance)
                                   +--> Reconciliation queue

```

Component	Owner	Module
WFM Aggregator	Operations	services/wfm_aggregator.py + services/csv_ingest.py
Leave Ledger	HR	services/leave_service.py
Rate Card Service	HR / Contracts	services/rate_card_service.py
Billing Engine	Finance	services/billing_engine.py (pure)
Reconciliation Service	Finance + Operations	services/reconciliation_service.py

The Billing Engine never queries upstream systems mid-computation. It receives a fully hydrated *HourlyInputs* + *AgentRateInputs* + *LeaveResolution* and returns a deterministic *BillingResult*. The orchestrator `services/finance_service.py` is the only module that touches multiple repositories at once.

3. The Seven-Step Pipeline (PDF Section 5.1)

Step	Operation	Function
1	Classify worked hours -> regular / OT / night / holiday	step1_classify_worked_hours
2	Resolve leave consumption (three policies)	leave_service.resolve
3	Apply rate multipliers (holiday + OT non-stacking by default, configurable)	step3_apply_rate_multipliers
4	Sum and accrue -> daily_gross	step4_sum_and_accrue
5	Roll up to monthly window	finance_service._month_to_date_gross
6	Apply min/max -> delta_reason + HR/Ops alerts	step6_apply_min_max
7	Persist with content_hash + provenance + parent_id chain	finance_service.process_breakdown

3.1 Worked example (PDF Section 5.2)

Encoded as a golden test
(tests/test_billing_engine.py::test_section_5_2_worked_example_golden).

```

regular_pay      = 168 x INR 250      = INR 42,000
overtime_pay     = 12  x INR 250 x 1.5 = INR  4,500
holiday_pay      =  8  x INR 250 x 2.0 = INR  4,000
night_premium    = 20  x INR 250 x 0.10 = INR   500
-----
              monthly_gross           = INR 51,000

min/max check   : INR 35,000 < INR 51,000 < INR 70,000 -> pass through
Final monthly_pay      = INR 51,000

```

3.2 Live verification

Smoke test against the running server using the worked-example agent:

```

curl -X POST http://localhost:8000/finance/breakdown -H "Content-Type: application/json" \
  -d '{"agent_id":"WORKED_EXAMPLE","date":"2026-04-14",
      "productive_minutes":480,"available_minutes":0,"break_minutes":0,
      "training_minutes":0,"scheduled_minutes":480,"worked_minutes":510,
      "overtime_minutes":30,"is_holiday":false,"night_minutes":60}'

-> 200 OK
daily_gross      : 2212.5000
regular_pay      : 2000.0000
overtime_pay     :  187.5000
night_premium    :   25.0000
content_hash     : 786292e467efabb7...

```

4. Leave Consumption - The Critical Edge Case (PDF Section 6)

Three explicit policies on each LeaveLedger row, set per agent at contract time. All three are implemented in `services/leave_service.py` and individually tested.

Policy	Behaviour	HTTP outcome
BLOCK (6.1)	Absence rejected; HR override required.	409 Conflict + BLOCK_POLICY_TRIGGERED
UNPAID_AUTO (6.2)	Recorded as LWP at zero rate; daily HR alert.	200 + billing_status = UNPAID_LWP
MANAGER_APPROVAL (6.3)	Queued as PendingApproval; billing not finalised.	200 + billing_status = PENDING_APPROVAL

4.1 Manager decision flow (PDF Section 6.3)

- POST `/reconciliation/pending-approvals/{id}/decision` with body `{manager_id, decision: 'APPROVE'|'DENY', note}`.
- APPROVE - treated as paid exception; ledger gets +1 approved and +1 consumed.
- DENY - reverts to unpaid.
- Section 6.3 month-close gate: while any PENDING_APPROVAL exists in the month, `monthly_rollup` appends a blocking alert and `days_counted` excludes those rows.

4.2 Daily Reconciliation (PDF Section 6.4)

Check	What it does	Threshold
Coverage	scheduled_minutes vs worked_minutes per agent	10% variance (configurable)
Leave-balance integrity	WFM absences vs Leave Ledger consumed (PDF Section 10)	0.5 leave-day drift
Preview vs actual	Preview OT cost vs actual recorded OT cost (PDF Section 10)	5% drift
Pending close block	Surfaces any PENDING_APPROVAL rows for the month	BLOCK severity

5. The WFM <-> Finance Contract (PDF Section 7)

Endpoint	Section	Notes
POST <code>/finance/breakdown</code>	7.1 + 7.2	WFM publishes daily-finalised event; Finance returns cost preview.
POST <code>/finance/breakdown/from-wfm-raw</code>	7.1 + 11	Accepts a raw <code>{status: minutes}</code> map; aggregator + pipeline run in one call.
POST <code>/finance/preview</code>	7.3	SYNCHRONOUS OT preview - <code>estimated_additional_cost</code> , <code>cap_breach</code> , <code>approval_recommendation</code> .

Endpoint	Section	Notes
GET /finance/breakdown/{agent}/{date}	-	Read-back for the Java dashboard.
GET /finance/breakdown/{agent}?start=&end;=	-	Date-range read.
GET /finance/monthly/{agent}/{year}/{month}	5.1 step 5 + 6	Month-close rollup with min/max applied to ACTUAL monthly_gross.

6. Streaming CSV Ingest

WFM data lives in Agent_Breakdown_140426.csv and is consumed row-by-row so memory stays bounded for arbitrarily large extracts. Each row is bucketised per (agent_id, local_date), routed through the Section 11 Status -> Pay Category mapping, then handed to the same seven-step Billing Engine.

CSV row

```
--> normalise headers / parse local interval start
--> accumulate per (agent_id, work_date)
--> wfm_aggregator.aggregate_status_map (Section 11)
--> finance_service.process_breakdown (Section 5.1 - all 7 steps)
--> FinancialBreakdown row + content_hash + provenance (Section 8)
```

6.1 Driving the ingest

Mode	Endpoint / Command
Server-side file by path	POST /ingest/csv/path?path=...
Multipart upload from frontend	POST /ingest/csv/upload (form field 'file')
Live progress feed (SSE)	GET /ingest/csv/stream
Dry-run preview	GET /ingest/csv/preview?limit=20
CLI / cron batch	python -m app.cli ingest [--path PATH] [--auto-create]

6.2 Column mapping

CSV header	Used as
Agent Id	agent_id
Status	WFM status (Section 11 routing)
Vendor Site	site - drives per-site holiday calendar
local_interval_sta (truncated in source)	local date for the agent-day bucket
Local Start Time	activity start - night-window check
time_in_interval_m	minutes in this status / interval

6.3 Smoke test against the real CSV

```
$ python -m app.cli ingest --auto-create
```

```
[OK] 61581666459416 2026-04-12 site=Teleperformance-Dublin rows= 66 gross=93.80
[OK] 61581666459416 2026-04-13 site=Teleperformance-Dublin rows= 64 gross=135.27
[OK] 61581666459416 2026-04-14 site=Teleperformance-Dublin rows= 18 gross=25.29
...
```

Summary:

```
          OK: 42
        BLOCKED: 0
  SKIPPED_UNKNOWN_AGENT: 0
          ERROR: 0
```

```
Aggregate parser sees:
  unique agents : 20
```

```
unique days    : 2026-04-12 / 2026-04-13 / 2026-04-14  
total minutes  : 11,728.7  
night minutes  : 1,028.5
```

7. Auditability & Compliance (PDF Section 8)

Every FinancialBreakdown row carries:

- **provenance** - full input snapshot (hours + rates + leave resolution + step1 buckets + settings).
- **content_hash** - SHA-256 of canonicalised provenance + components.
- **parent_id** - when a correction lands (WFM re-publishes worked_minutes), the new record points to the original. The original is preserved - mitigates Section 10 *'Backfill of corrections poisons month-end'*.

In addition, the AuditLog captures BILLING_COMPUTED, BILLING_BLOCKED, MANAGER_DECISION, and LEAVE_BALANCE_ADJUSTED events with actor + timestamp.

8. Risk Mitigations Implemented (PDF Section 10)

Risk	Mitigation	Where
Stale rate cards produce wrong cost previews	version_id + 24h TTL flag in preview response	rate_card_service.is_stale
WFM and Leave Ledger drift apart	Daily reconciliation; alert if drift > 0.5 day	reconciliation_service._leave_integrity_check
Holiday calendar mismatch across regions	Per-site calendar; multi-site agents resolve by primary_site	HolidayCalendar + finance_service._resolve_holiday
Overtime gaming (clock-in early/late)	Explicit OT requires WFM-approved overtime_minutes; un-approved -> regular rate	billing_engine.step1_classify_worked_hours
Min-pay floor activations hidden from HR	Every floor activation emits a MIN_FLOOR_APPLIED alert	billing_engine.step6_apply_min_max
Backfill of corrections poisons month-end	New record + parent_id; original preserved	finance_service.process_breakdown

9. Test Results

```
$ python -m pytest -q
..... 26 passed in 0.19s
```

Suite	Coverage
test_billing_engine.py	Steps 1, 3, 6 individually + full compute + Section 5.2 golden case
test_leave_policies.py	BLOCK (6.1), UNPAID_AUTO (6.2), MANAGER_APPROVAL (6.3) + decide_pending
test_reconciliation.py	Coverage variance threshold + pending-approval blocks month-close
test_status_mapping.py	Section 11 appendix - productive / paid_idle / shrinkage / training / unmapped
test_csv_ingest.py	Per-status aggregation + multi-day split + night-window + agent filter + blank-row handling + BOM tolerance

10. How to Run & Test

Setup

```
cd c:\mudrik\mvp_ireland\finance\backend
pip install -r requirements.txt
python seed_data.py          # loads Section 5.2 worked-example + CSV sample agents
python run.py                # starts uvicorn on :8000
```

Quick sanity loop

```
python -m pytest -q          # 1. unit + integration tests
python seed_data.py          # 2. seed the DB
python -m app.cli ingest --auto-create # 3. real CSV through end-to-end
python run.py                # 4. open http://localhost:8000/docs
```

Where to point your browser

URL	What it is
http://localhost:8000/docs	Swagger UI - click 'Try it out' on any endpoint
http://localhost:8000/redoc	ReDoc - read-only API browser
http://localhost:8000/openapi.json	OpenAPI 3 spec - feed to openapi-generator
http://localhost:8000/health	Liveness JSON

Note: 0.0.0.0 in the uvicorn output is a bind address (listen on all interfaces); browse to localhost or 127.0.0.1.

11. Design Doc Coverage Matrix

Every section of WFM_Finance_Integration_Design.pdf v0.1 is mapped to a concrete artefact in this implementation.

PDF Section	Title	Implemented in
1	Problem Statement	Architecture addresses all 3 failure modes - see Section 2 above.
2.1	Goals - In Scope	All 5 in-scope items implemented (per-agent billing, min/max, leave, preview, daily/weekly/monthly cycles).
2.2	Out of Scope (v1)	Explicitly NOT implemented; documented in README.
3.1	Component View	5 components mapped in services/ (see Section 2 of this report).
3.2	Data Flow	Deterministic chain in finance_service.process_breakdown.
4.1	Agent Master	models.Agent + routes /agents (CRUD).
4.2	Leave Ledger	models.LeaveLedger + routes /leaves (CRUD + adjust).
4.3	Hourly Breakdown Input	schemas.HourlyBreakdownIn - matches PDF field set 1:1.
5.1	Seven-Step Pipeline	services/billing_engine.py - all 7 steps as individual functions.
5.2	Worked Example	Golden test test_section_5_2_worked_example_golden + live curl smoke.
6.1	Policy: BLOCK	leave_service - returns 409 BLOCK_POLICY_TRIGGERED.
6.2	Policy: UNPAID_AUTO	leave_service - billing_status = UNPAID_LWP + HR alert.
6.3	Policy: MANAGER_APPROVAL	PendingApproval queue + /reconciliation/pending-approvals/{id}/decision.
6.4	Daily Reconciliation	3 checks in reconciliation_service.run + pending close block.
7.1	What WFM publishes	POST /finance/breakdown - body matches PDF JSON shape.
7.2	What Finance publishes back	schemas.BillingResponse - matches PDF response shape.
7.3	Operations cost-preview endpoint	POST /finance/preview - synchronous, returns approval_recommendation.
8	Auditability & Compliance	content_hash on every row + parent_id chain + AuditLog table.
9	Rollout Plan	Phases 0-4 mapped in README; engine isolated for Phase 0 unit tests.
10	Risks & Mitigations	All 6 risks have explicit mitigations - see Section 8 above.
11	Appendix: Status -> Pay Category	utils/status_mapping.py + GET /status-mapping - all 8 statuses.

Document generated by backend/generate_report.py. Re-run after each significant change to keep the audit trail current.